Lab Assignment No.	02
Title	Design a distributed application using RMI for remote computation where client submits two strings to the server and server returns the concatenation of the given strings.
Roll No.	
Class	BE AI & DS
Date Of Completion	
Subject	Computer Laboratory III[417534]
Assessment Marks	
Assessor's Sign	

Assignment No.2

Title: Design a Distributed Application Using RMI for Remote String Concatenation

Problem Statement: Develop a distributed application using Remote Method Invocation (RMI) where a client submits two strings to a remote server. The server concatenates the given strings and returns the result to the client program. The application should demonstrate the concept of distributed computing by enabling remote execution of string concatenation.

Prerequisites: Basic understanding of distributed systems, RMI mechanism, client-server architecture, string manipulation, programming in Python, and network communication fundamentals.

Software Requirements: Windows/Linux/MacOS, Python, RPyC (Remote Python Call) library, IDE (VS Code, PyCharm), and networking tools (Wireshark, Telnet, Netcat).

Hardware Requirements: Intel Core i3 or higher, 4GB RAM, 100MB free disk space, and network connectivity for client-server communication.

Theory:

Introduction to Remote Method Invocation (RMI) in Python:

Remote Method Invocation (RMI) enables a program to call methods on remote objects hosted on another machine. In Python, this can be implemented using the **RPyC (Remote Python Call)** library, which simplifies communication between distributed applications by enabling remote execution of functions.

Working of RMI in Python:

1. The client sends a request to the server with two input strings.
2. The server processes the request, concatenates the given strings, and prepares the response.
3. The server sends the concatenated result back to the client.
4. The client receives the result and processes it accordingly.

String Concatenation:

String concatenation is the process of joining two or more strings together. In Python, it can be performed using:

- The `+` operator: `result = str1 + str2`
- The `join()` method: `result = "".join([str1, str2])`

For example:

- **Input:** "Hello" and "World"
- **Output:** "HelloWorld"

Implementation Approach:

To implement an RMI-based distributed string concatenation system in Python, the following approach is used:

1. **Client Program:** Sends two strings to the server via an RMI request.
2. **Server Program:** Receives the request, concatenates the given strings, and sends the result back to the client.
3. **Communication Layer:** Handles the exchange of requests and responses between the client and server using the RPyC library.

Advantages of RMI-Based Computation in Python:

- Simplifies distributed computing by allowing remote function execution.
- Reduces computational load on client devices by offloading processing to the server.
- Facilitates modular and scalable application design.
- Provides seamless communication between Python applications over a network.

Source Code –

Client.py

```
import Pyro4

# Get the server object using its URI
uri = input("Enter the server URI : ")
string_concatenator = Pyro4.Proxy(uri)

# Get inputs string from the user
str1 = input("Enter first string : ")
str2 = input("Enter second string : ")

# Call the remote method on the server
result = string_concatenator.concatenate(str1, str2)

print("Concatenated String : ", result)
```

Output

Enter the server URI :

PYRO:obj_3f70c6b68e9e4712a73a4167337c1e58@localhost:61368

Enter first string : Hello

Enter second string : World

Concatenated String : Hello World

Server.py

```
import Pyro4

@Pyro4.expose
class StringConcatenator(object):
    def concatenate(self, str1, str2):
        return str1 + " " + str2

# Register the StringConcatenator class as a Pyro Object
daemon = Pyro4.Daemon()
uri = daemon.register(StringConcatenator)
```

```
print("Server URI : ", uri)

# Start the server
daemon.requestLoop()
```

Output –

Server URI : PYRO:obj_3f70c6b68e9e4712a73a4167337c1e58@localhost:61368

Conclusion: This practical demonstrates the efficiency of RMI in distributed computing by remotely executing string concatenation in Python. It highlights modular design, network communication, and seamless integration in client-server architectures.